Terminologies :-

Search - step by step procedure to solve a problem within a given search space

Search space - set of all possible solutions

Start state - the initial point where the search begins

Goal ~~state~~ test - A function that checks if the current state is the goal state

Search tree - A tree that represents the search problem, with the root node corresponding to the initial state

Actions - All available moves the agent can take

Transition Model - describes the outcome of each action

Path cost - A function that assigns a cost to each path

solution - An action sequence from start node to goal

optimal solution - the sol'n with the lowest path cost among all possible solutions

A Heuristic function is admissible if it never over estimates the true cost of reaching the goal

$h(n) \leq h(n)^*$ ↓ true cost $h(n)$ - heuristic cost [estimated cost to reach goal from node n]

the heuristic search is a search algorithm that uses a heuristic function to guide its search process towards the goal more efficiently. It has some key components:

- heuristic value $h(n)$
- open List :- stores the nodes that are yet to be expanded
- closed List - stores " " " already been expanded

*

Types of search Algorithms

1. uninformed (Blind) search :-
   - Lacks domain specific knowledge or heuristics
   - uses brute force, only knows how to traverse the tree and identify goal nodes
   - Ex:- BFS, DFS, uniform cost search

2. Informed (Heuristic) search:
   - Utilizes domain knowledge to guide the search
   - uses problem-specific info to explore the most promising paths

   Ex:- A*, Greedy best-first search

Heuristic function → estimates how close the current state is to the goal, guiding the search process

→ heuristic methods may not always guarantee the optimal solution, they provide a good solution within reasonable time

Breadth first search (BFS): (uninformed search)

→ BFS is a fundamental algorithm used for traversing or searching through tree or graph structures

Key features

→ BFS explores all nodes at present depth level before moving on to nodes at the next depth level. This characteristic makes it particularly effective for finding the shortest path in unweighted graphs

→ The algorithm starts at the root node and expands outward to neighboring nodes.

→ BFS is typically implemented using a FIFO queue.

Steps

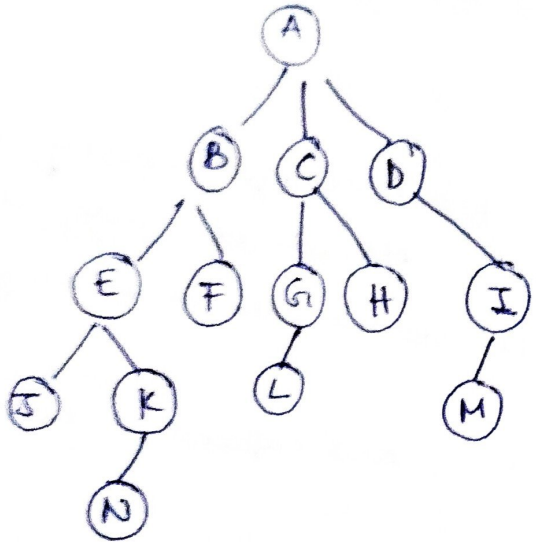
1. Initialization → Start by placing the root node in the queue
2. Node Expansion → While the queue is not empty:
   → Dequeue a node from the front the queue
   → If this node is the goal node, return the path to the goal
   → Enqueue all of its successors (children) to the back of the queue.
3. Continue till the queue is empty or the goal is found.

Advantages:
1. Completeness [guarantees that if a sol'n exists, it will be found]
2. optimality [finds the shortest path given that all edges have the same weight / cost]

disadvantages
1. Memory consumption
2. Time complexity.

Applications
1) Pathfinding Algorithms
2) Network broadcasting
3) puzzle solving

Example



Goal

Queue

← [A] ←

~~B~~ C D

~~C~~ D E F

~~D~~ E F G H I

~~E~~ F G H I

~~F~~ G H I J K

~~G~~ H I J K

~~H~~ I J K L

~~I~~ J K L

~~J~~ K L M

~~K~~ L M

~~L~~ M N

~~M~~ N

(N) → Goal

space & Time complexity :- $O(b^d)$

b → branch factor

d → depth of shallowest solution

Depth first (limited) search :-

→ Variant of DFS search with a depth limit

→ Nodes are expanded up to a predefined depth limit, preventing ∞ paths

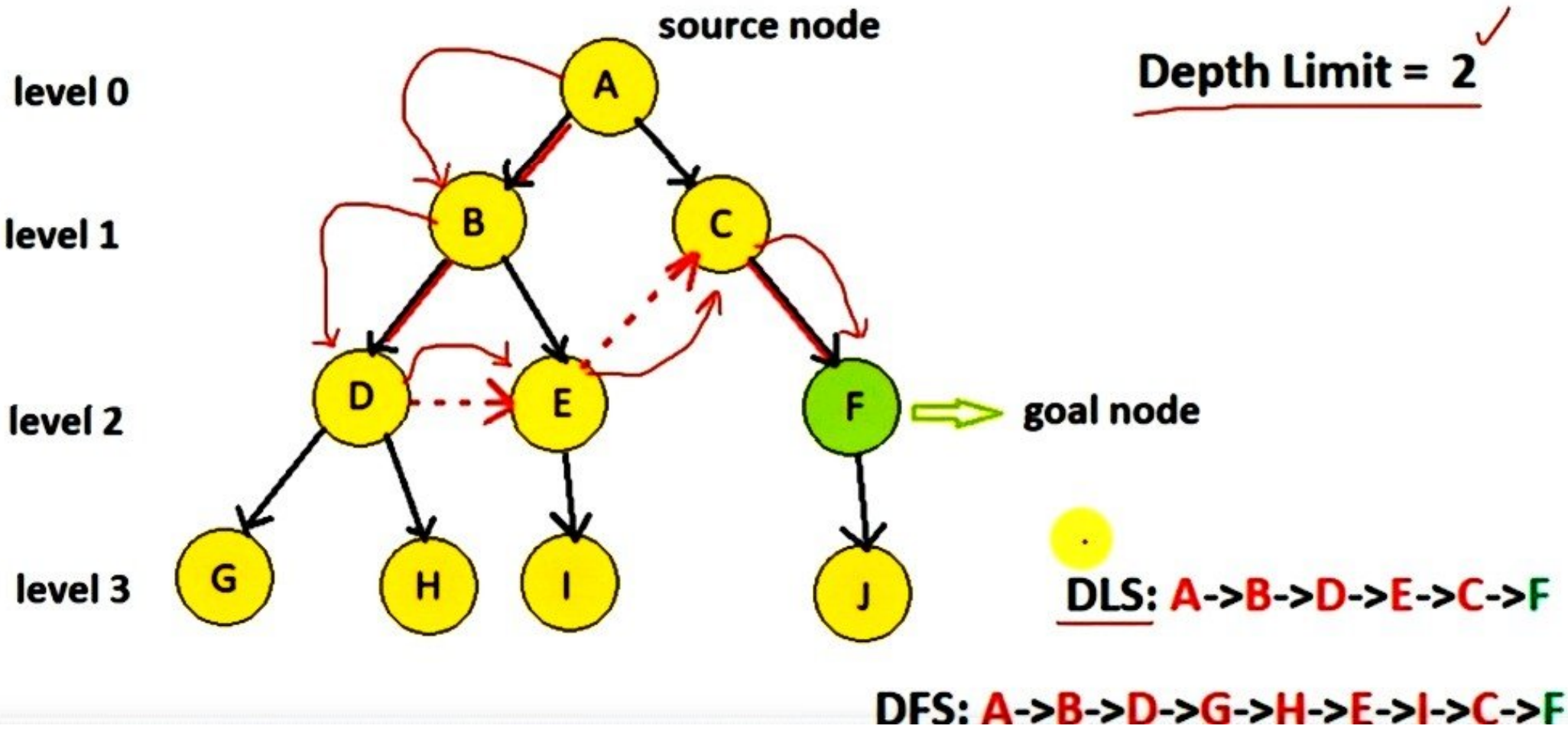
Failure conditions

- No solution exists within the search space
- No solution found within the depth limit

Steps

1. Start from the root node
2. Traverse nodes upto the depth limit
3. Treat nodes at the depth limit as having no successors
4. If no solution is found, return a failure

Depth Limited Search Algorithm in Artificial Intelligence



Advantages:

→ Memory efficient

disadvantages

1. Incomplete : May miss solutions beyond the depth limit
2. Non-optimal : Not guaranteed to find the best soln if multiple exist

Time & Space complexity (for DFS & Depth limit search)

Time – $O(b^l)$ b → branching factor

Space – $O(b \times l)$ l → depth limit

Types of search Algorithms

1. Uninformed (Blind) search :-
   - Lacks domain specific knowledge or heuristics
   - uses brute force, only knows how to traverse the tree and identify goal nodes
   - Ex :- BFS, DFS, uniform cost search

2. Informed (Heuristic) search:
   - utilises domain knowledge to guide the search
   - uses problem-specific info to explore the most promising paths

   Ex :- A*, Greedy best-first search

Heuristic function → estimates how close the current state is to the goal, guiding the search process

→ heuristic methods may not always guarantee the optimal solution, they provide a good solution within reasonable time

Uniform - Cost Search Algorithm (UCS) [Uninformed search]

→ The uniform - cost search (UCS) algorithm is a search strategy used for finding the least - cost path in a weighted graph or tree

Key features:-

→ UCS is specifically designed to handle graphs or trees where each edge has a different cost, allowing it to find the optimal path based on cumulative costs.

→ The algorithm expands nodes based on their total path cost from the root node, ensuring that at each step, the least expensive option is chosen

→ Priority Queue Implementation: UCS typically uses a priority queue to efficiently manage and select the next node to explore, prioritizing nodes with the lowest cumulative costs

Steps:-

1. Initialization:- Start by placing the root node in a priority queue with a cost of zero.

2. Node Expansion:- While the priority queue is not Empty:

→ remove the node with the lowest cost from the queue

→ If this node is the goal, return the path and its cost

→ Generate the successors of the current node and calculate their cumulative costs.

→ Add these successors to the priority queue.

3. Repeat: Continue the process until the goal is found or the queue is empty

Advantages:

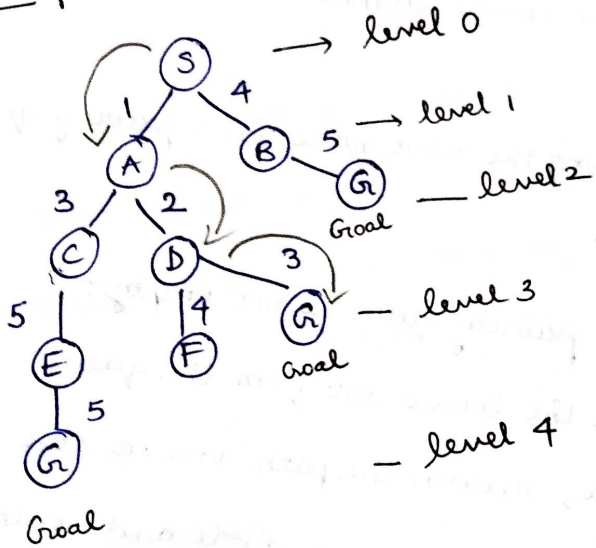
1. UCS is guaranteed to find the least-cost path, making it optimal for cost based problems (optimality)
2. UCS will find a solution if one exits provided the search space is finite

disadvantages:

① UCS only considers path cost, which means it may not find a solution with least no. of steps if higher-cost paths lead to better sol'ns

② It can get stuck in ∞ loops if there are cycles with 0 or -ve costs

Example:-



Path with min cost :- S → A → D → G cost = 6

Space & Time complexity

$O(b^{1+[c^*/\epsilon]})$

b : branching factor

c^* - cost of the optimal solution

ϵ - min cost of a step

Informed Search Algorithms :-

1. Best First search Algorithm (Greedy search)
2. A* search Algorithm

Best - First search Algorithm

→ The greedy best-first search algorithm selects the most promising path based on a heuristic function at each step, aiming to find the optimal solution efficiently

Key features :-

→ It combines aspects of both depth first search (DFS) and BFS

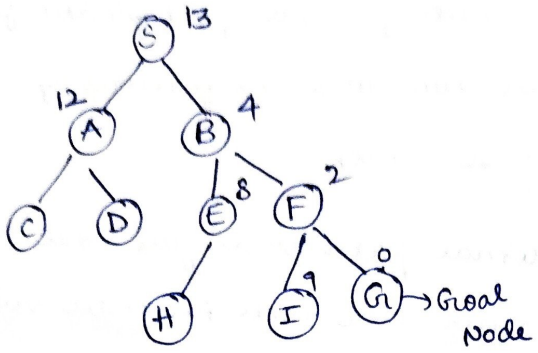
→ Utilizes a heuristic function to estimate the cost to reach the goal, guiding the search process

→ Expands the node that appears closest to the goal based on heuristic values

Steps

1. Initialization: Place the starting node into the OPEN list
2. Check OPEN list : if the open list is empty, Stop & return failure
3. Node selection : Remove the node 'n' with the lowest heuristic value h(n) from the open list and place it in closed list
4. Node expansion : expand node 'n' and generate its successors
5. Goal check : check if any successor is the goal node. If found, return it & stop
6. Successor Evaluation : for each successor, evaluate its function f(n) If the node has not been in the OPEN list or closed list, add it to the open list
7. return to step 2

Example :-



node	h(n)
S	13
A	12
B	4
C	7
D	3
E	8
F	2
H	4
I	9
G	0

Step-1 initialization

open [A, B] close [S]

iteration 1

open [A], close [S, B]

iteration 2

open [E, F, A], close [S, B]

open [E, A], close [S, B, F]

iteration 3

open [E, A, I, G], close [S, B, F]

open [E, A, I], close [S, B, F, G]

the path is S → B → F → G.

Time complexity

$O(b^m)$ – Time & space

b - branching factor

m - max depth

Advantages:-

- More efficient than BFS and DFS due to its heuristic guidance
- More flexible

disadvantages: 1) The algorithm is incomplete, even for finite state spaces

2) it is not optimal

Applications :- 1) Used in GPS systems and navigation software to find the shortest route b/w locations

2) Network routing

3) puzzle solving – to find optimal moves

A* Search Algorithm

→ The A* search algorithm is a widely used pathfinding and graph traversal algorithm that combines the features of Uniform Cost Search and Greedy best first search.

→ It effeciently finds the optimal path to a goal node by evaluating nodes based on their cost and heuristic values

Algorithm :-

1. Initialization : place the starting node in the OPEN list
2. check open list : If the list (open) is empty, return failure & stop
3. select node : choose the node with the smallest evaluation function $f(n) = g(n) + h(n)$. If this node is the goal node, return success and stop.
4. Expand node: Generate all successors of the selected node and place it in the CLOSED list. For each successor n':
   - If n' is not in OPEN or CLOSED, compute its evaluation function and add it to the OPEN list
   - If n' is already in open or closed, attach it to the back pointer reflecting the lowest $g(n')$ value
5. repeat the process until the goal is found.

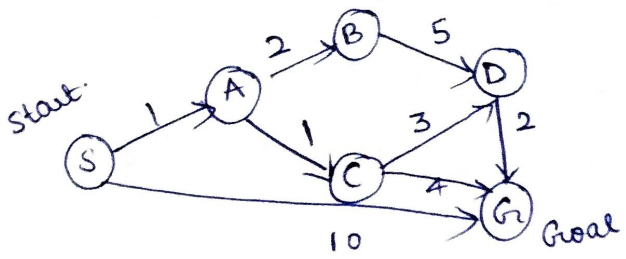
Advantages :-

1. A* is optimal and complete
2. Efficiently solves complex problems
3. Expands fewer nodes compared to other search algorithms

disadvantages :-

1. Memory- intensive as it keeps all generated nodes in memory, making it impractical for large-scale problems
2. The heuristic may lead to non-optimal paths in some cases

Example:-



State	h(n)
S	5
A	3
B	4
C	2
D	6
G	0

① $S \rightarrow A \rightarrow F(n) = g(n) + h(n)$
$= 1 + 3 \quad h(A)$
$= 4$ ✓

$S \rightarrow G = F(n) = 10 + 0$ ×
$= 10$ (hold)

Time complexity
$O(b^d)$ time & space

② $S \rightarrow A \rightarrow B \Rightarrow F(n) = 3 + 4 = 7$ (hold) ×

$S \rightarrow A \rightarrow C \Rightarrow F(n) = 2 + 2 = 4$ ✓

③ $S \rightarrow A \rightarrow C \rightarrow D \Rightarrow F(n) = 5 + 6 = 11$ (hold) ×

$S \rightarrow A \rightarrow C \rightarrow G \Rightarrow F(n) = 6 + 0 = 6$ ✓

$S \rightarrow A \rightarrow C \rightarrow G$ final path cost = 6

Applications

1. Pathfinding in Games
2. Robotics
3. Network routing

Memory Bounded A* (heuristic) / Simplified Memory bounded A*

→ SMA* is a shortest path algorithm that is based on the A* algorithm

→ The difference b/w SMA* and A* is that SMA* uses a bounded memory, while the A* might need exponential memory

key features

→ SMA* uses a fixed limit on memory usage, making it suitable for environments with limited resources. ~~In contrast,~~

→ Similar to A*, SMA* expands the most promising nodes based on the evaluation function $f(n) = g(n) + h(n)$ where:

$g(n)$: the cost to reach node n from the start node

$h(n)$: heuristic

→ When the memory is full, SMA* will prune the least promising nodes from the open list, allowing it to keep the most promising branches in memory.

→ This is done while ensuring that the search remains complete and can find optimal paths when possible

Advantages :

1) SMA* effectively balances memory constraints with search effeciency, making it more practical in environments with limited memory

2) SMA* can find optimal solutions under certain conditions, similar to A*, but with reduced memory overhead.

Local search strategies:

→ These are optimization algorithms that improve a solution by exploring neighbouring states.

→ These focus on local improvements without considering the entire search space

i) Hill climbing Algorithm

→ A local search algorithm that continuously moves towards increasing values to find the peak solution.

→ It terminates when no neighbouring state has a higher value

→ used for optimizing mathematical problems

Features:

1. Generate and test Variant : It is a variation of the "generate and test" method, using feedback to guide the search
2. Greedy Approach: Always moves towards a direction that improves the cost or value. [only considers the immediate neighbouring states]
3. No backtracking: It doesnot revisit previous states, thus lacks memory of prior steps.

State-space Landscape Regions

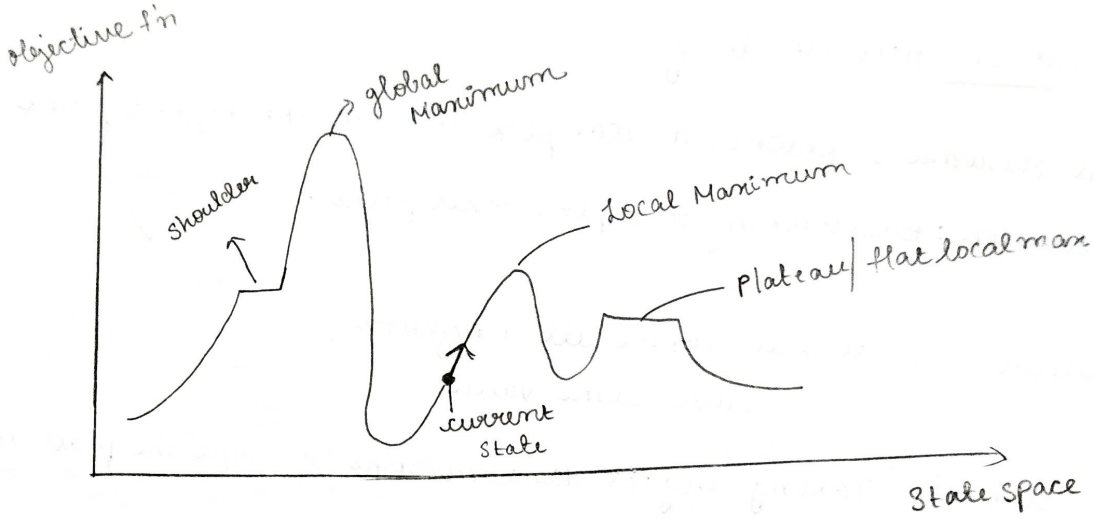
i) Local Maximum : A peak better than its neighbours, but not the highest. [has the best cost function]

ii) Global Maximum : the best possible solution

iii) Current State : the present position in the search space

iv) Flat Local Maximum / plateau: flat region where all neighbours have the same value.

v) Shoulder : A plateau region with an uphill edge



Hill climbing Algorithm types:

1. Simple Hill climbing:
   - Easiest variation of Hill climbing
   - Evaluate initial state, loop through applying operators and check if the new state is the goal
   - Less time consuming, less optimal
   - it examines neighboring points one at a time and selects the one which optimizes the current cost as the next state

2. Steepest - Ascent Hill climbing:
   - first all the neighboring points are examined and the point closest to the solution is taken as next state
   - This is more time consuming

3. Stochastic Hill climbing
   - It does not examine its neighbors before moving.
   - It selects a random neighbor and decides whether to update it as current state or not

Problems in Hill climbing

1. local Maxima : Reaches a local peak but not the highest point

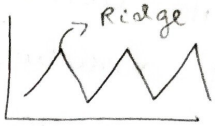
   solution : Backtracking to explore other paths.

2. Plateau : Flat area where all neighbors have same value

   solution : Taking big or random steps to escape the plateau

3. Ridges : A slope that can't be climbed in one move.

   solution : Bidirectional search or moving in different directions



Applications

→ used in problems which require evaluation/thinking

→ coordination between robots

→ circuit design

Algorithm

1. Evaluate the initial state. If it is also a goal state then return it, otherwise continue with the initial state as the current state

2. Loop until the solution is found or until there are no new operators to be applied in the current state

   a) select an operator that has not yet been applied to the current state and apply it to produce a new state

   b) Evaluate the new state, (with the help of heuristic function)

      i) it is a goal state, then return it & quit

      ii) if it is not a goal state but it is better than the current state, then make it as current state

      iii) if it is not better than the current state, then continue in loop

(ii) Simulated Annealing Algorithm:

→ A probabilistic optimization algorithm inspired by physical process of annealing, where materials are heated and slowly cooled to reach a stable, optimal structure

Key features:-

1. Efficiency and completeness: unlike the traditional hill climbing, simulated annealing can avoid getting stuck in local optima by allowing moves to worse states temporarily.

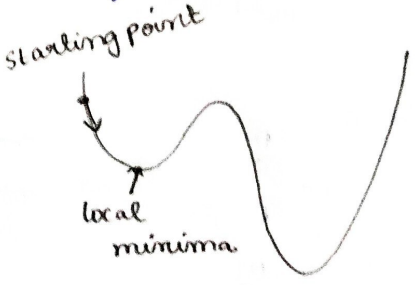
2. Mimics the process of gradually cooling a material. When heated, the structure is flexible and prone to change. As it cools, it stabilizes in a more optimal form

→ It is used for optimizing parameters in a model

→ Very useful for situations where there are a lot of local minima

→ This algorithm operates well for non linear objective functions where other local search algorithms fail

→ Helps avoid getting stuck in local minima by introducing randomness or 'shaking' in the current state



→ This shaking allows the algorithm to explore a larger area of the search space, increasing the chance of finding the global optimum

Applications:-

→ Parameter optimization in models

→ Problems with multiple local minima, like job scheduling, traveling salesman etc

iii) Local Beam Search Algorithm

→ It is a heuristic search algorithm that explores a graph by expanding the most promising nodes but limits the number of nodes stored at each level to a predefined "beam width"

Key features :-

1. Very similar to Breadth-first-search, but instead of expanding all nodes, it only keeps the best few at each level based on some evaluation function
2. Beam Width : A parameter that limits the no. of nodes expanded at each level. only the top 'K' nodes are kept, where 'K' is the beam width (if $K = 1 \Rightarrow$ Best first searching)
3. It balances b/w BFS (which explores all paths) and greedy search (which explores the best path but may miss better alternatives)

Steps:-

1. Start with the initial node
2. At each level, expand all the current nodes and evaluate them
3. Keep only the top 'K' nodes (based on beam width & discard the rest.
4. Continue expanding & pruning nodes at each level until the goal is reached or no nodes are left

Advantages

1. By limiting the no. of nodes, it reduces memory usage
2. Expands fewer nodes, making the search process quicker

disadvantages:

1. May miss the optimal solution if its outside the beam width

Applications :

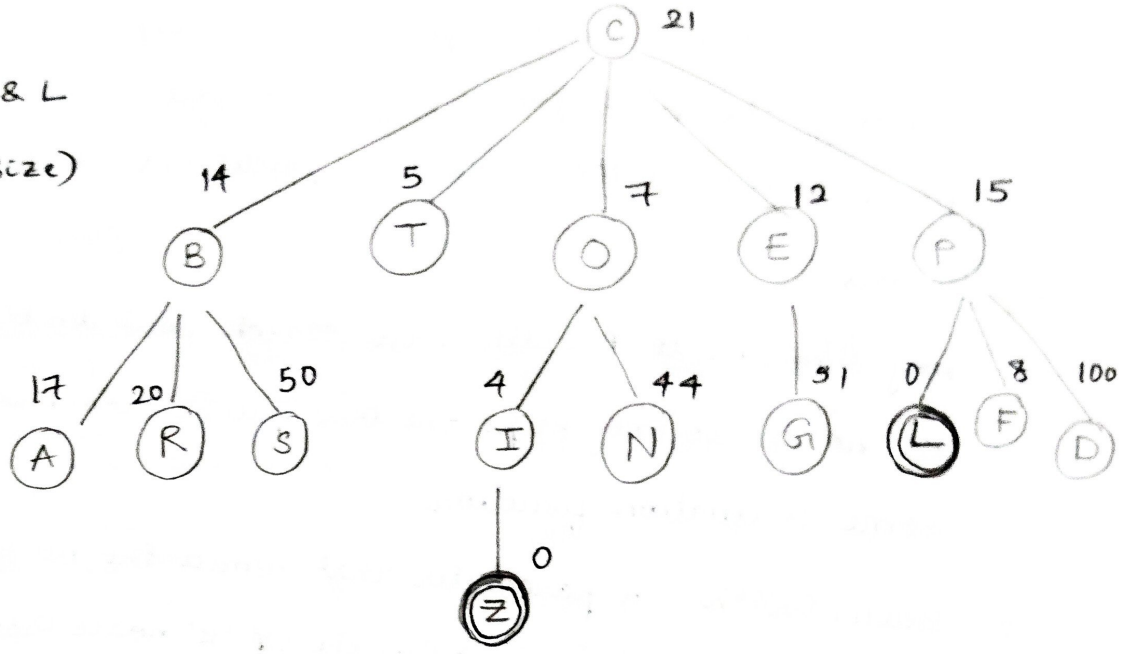
1) NLP → used for speech recognition,
2) Pathfinding [can effeciently search for the best moves]

EX:-

Start state : C

Goal state : Z & L

n = 2 (beam size)



Iteration 1

open List = {C}

Iteration 2 :- find successor of C = B T O E P
14 5 7 12 15

open List = {T, O} T, O have least distances (n=2)

Iteration 3 :-

T has no successors, remove T from open list

find successor of O, replace O with I and N in open list

open List = {I, N}

Iteration 4:

successor of I is (Z) Goal found.

iv) Genetic Algorithms (GA)

→ GA are a class of optimization algorithms inspired by the process of natural selection and genetics

→ They are used to find approximate solutions to complex problems by mimicking biological evolution

Key concepts :-

1. population - A set of candidate solutions (individuals) to the optimization problem
2. chromosomes - representation of individuals
3. Fitness Function - A function that evaluates how good each individual is at solving the problem.
   → It measures the quality of the solution
4. selection - the process of choosing individuals from the population to breed based on their fitness.
5. crossover - A genetic operator that combines parts of two parent chromosomes to create offspring.
6. Mutation - A genetic operator that introduces random changes to an individual's chromosome to maintain genetic diversity within the population

Advantages :-

1. Can handle a variety of optimization problems
2. Processes multiple solutions simultaneously, increasing efficiency
3. capable of exploring the solution space broadly, reducing the risk of getting trapped in local optima

Disadvantages

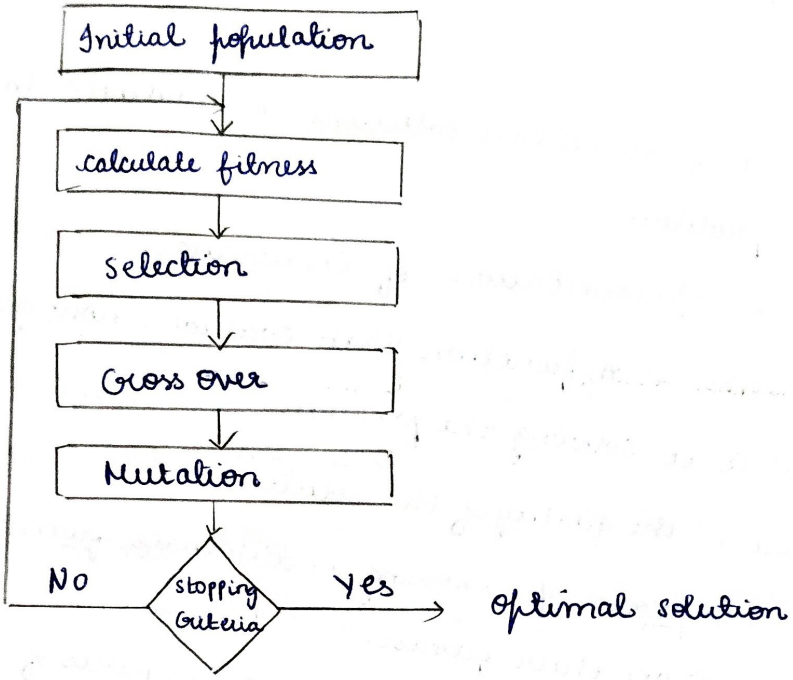
1. No guarantee of finding the optimal solution
2. May require many iterations to converge to a good solution

→ Solve complex problems

→ focus on optimization

→ from a group of individuals, the best will survive (survival of the fittest)

Algorithm :-

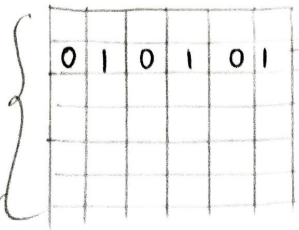


Steps of GA :-

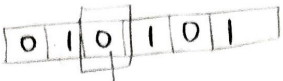
1. Generate an initial population of individuals randomly
2. Calculate the fitness of each individual in the population
3. select individuals for reproduction based on their fitness [some fitness function]
4. perform crossover on selected individuals to create new offspring
5. Apply mutation to some offspring to introduce variability
6. Replace the old population with the new offspring population
7. repeat the evaluation, selection, crossover, mutation steps until a stopping criterion is met.

Applications :- 1) useful for optimizing job scheduling and resource allocation

(Subset of all possible sol'ns)



Population



Chromosome (one sol'n)

gene (one element position of chromosome)

0 Allele (value of gene)

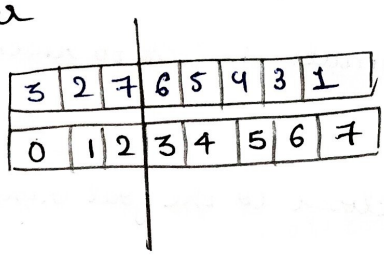
Phenotype
(Population in the real world sol'n Space)

Encode →

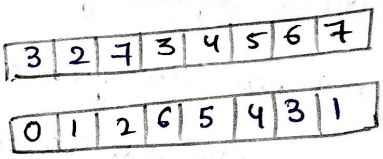
← decode

Genotype
(Population in the computation space)

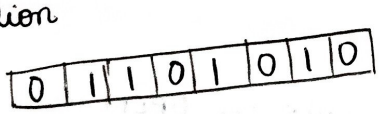
Crossover



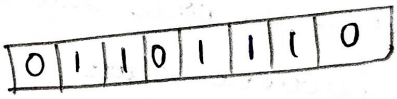
→



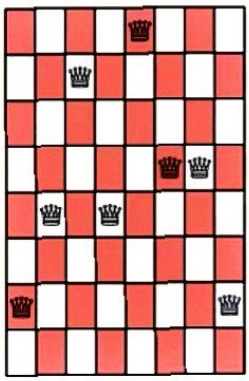
Mutation



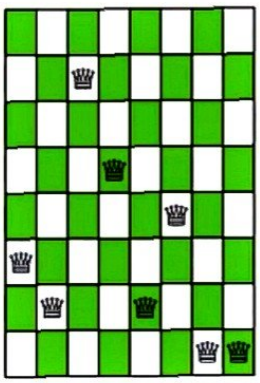
→



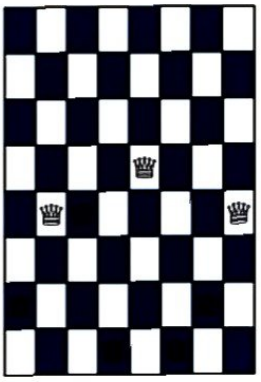
Example: 8 queens problem



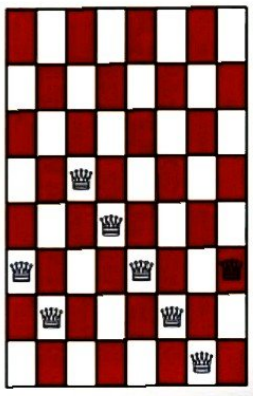
2	4	7	4	8	5	5	2



3	2	7	5	2	4	1	1



2	4	4	1	5	1	2	4

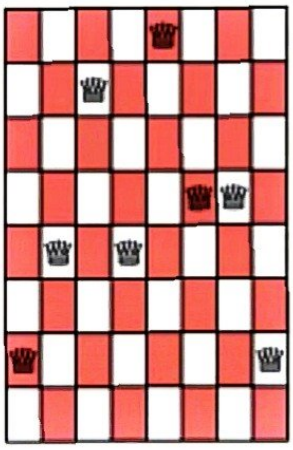


3	2	5	4	3	2	1	3

17:48

Calculating Fitness Function

- No. of non-attacking pairs

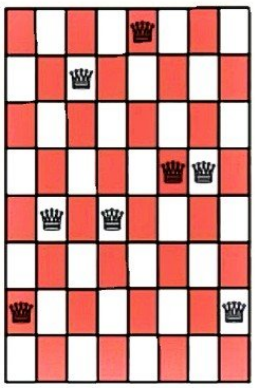


2	4	7	4	8	5	5	2

Q1 :6
Q2 :5
Q3 :4
Q4 :4
Q5 :3
Q6 :1
Q7 :1
Q8 :0

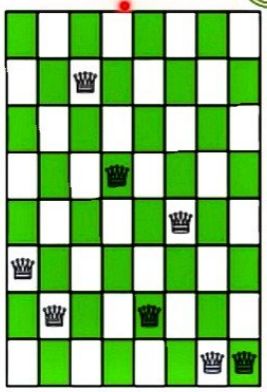
17:49

Selection based on the fitness value



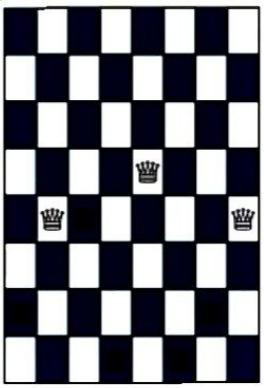
2	4	7	4	8	5	5	2

Fitness= 24
=24/(24+23+20+11)
= 24/78= 31 %



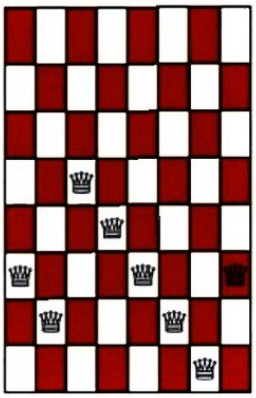
3	2	7	5	2	4	1	1

Fitness= 23
=23/(24+23+20+11)
= 23/78= 29 %



2	4	4	1	5	1	2	4

Fitness= 20
=20/(24+23+20+11)
= 20/78= 26 %

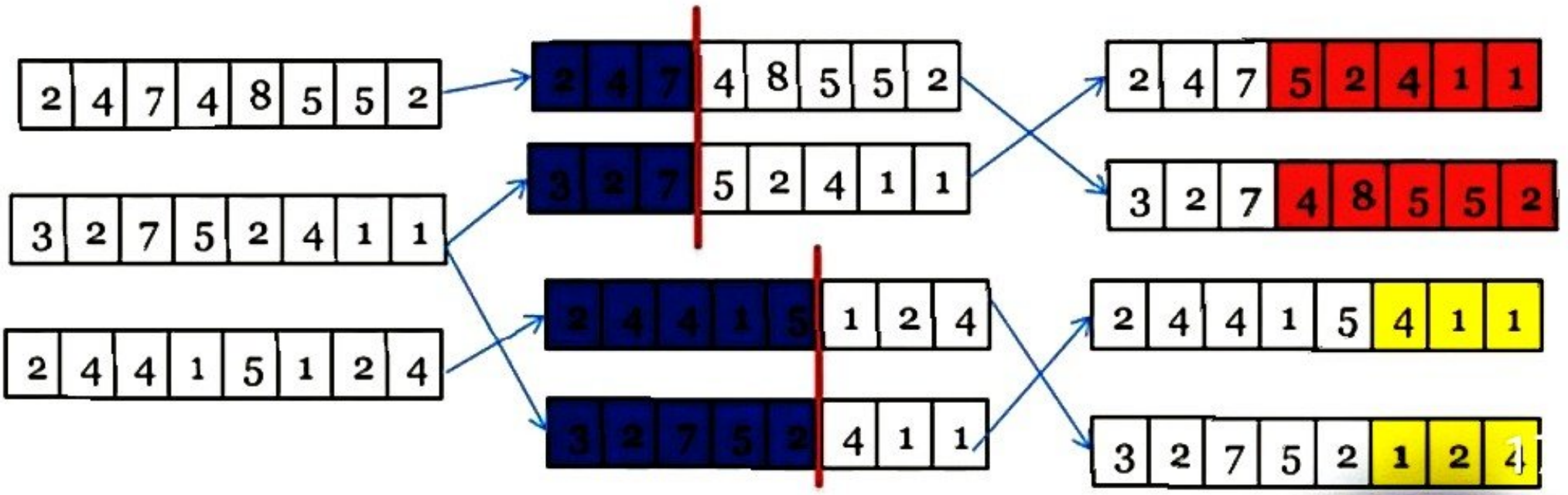


3	2	5	4	3	2	1	3

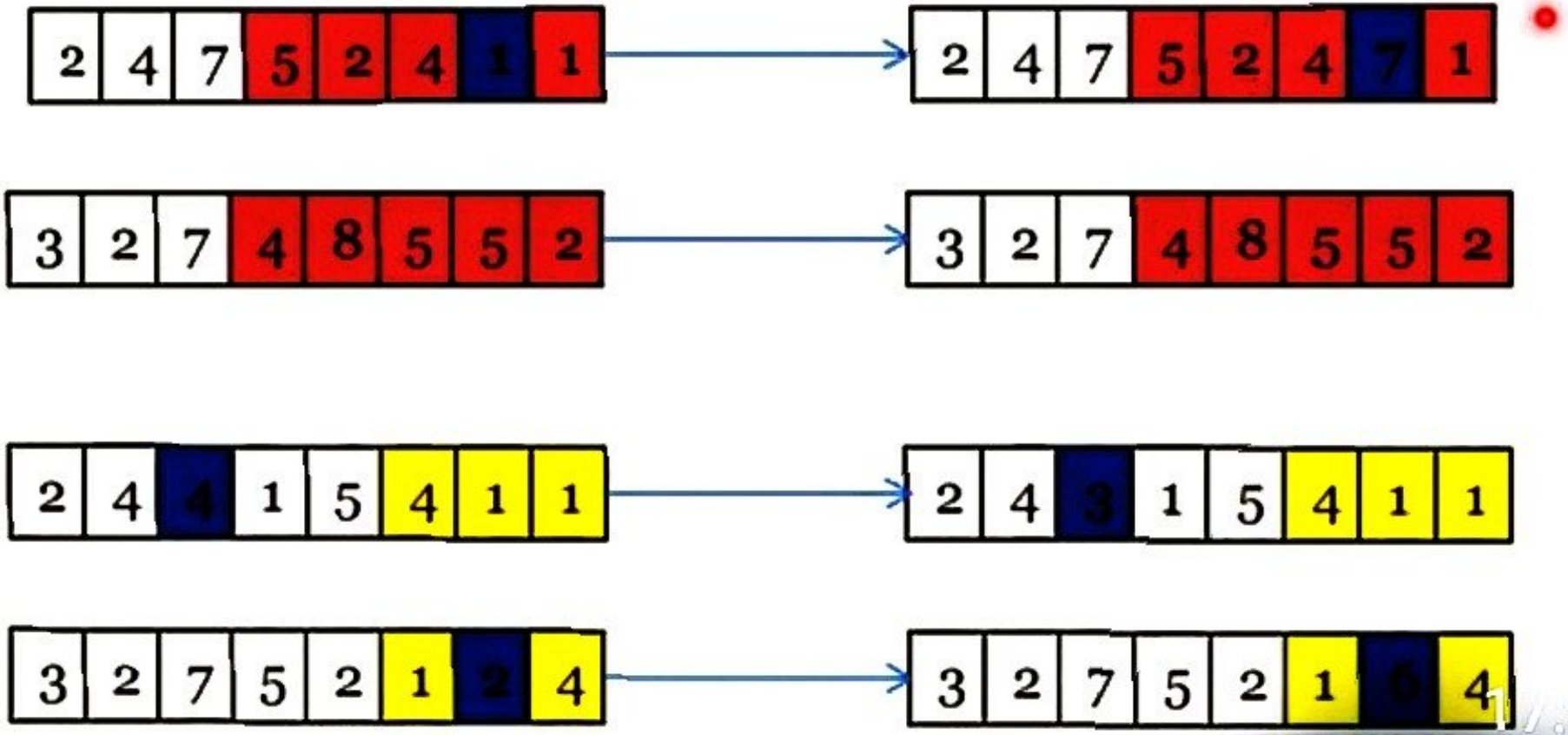
Fitness= 11
=11/(24+23+20+11)
= 11/78= 14 %

17:49

Crossover


2 4 7 4 8 5 5 2
3 2 7 5 2 4 1 1
2 4 4 1 5 1 2 4
2 4 7 4 8 5 5 2
3 2 7 5 2 4 1 1
2 4 4 1 5 1 2 4
3 2 7 5 2 4 1 1
2 4 7 5 2 4 1 1
3 2 7 4 8 5 5 2
2 4 4 1 5 4 1 1
3 2 7 5 2 1 2 4

Mutation


2 4 7 5 2 4 1 1
2 4 7 5 2 4 7 1
3 2 7 4 8 5 5 2
3 2 7 4 8 5 5 2
2 4 4 1 5 4 1 1
2 4 3 1 5 4 1 1
3 2 7 5 2 1 2 4
3 2 7 5 2 1 6 4